Quantum DNA Pattern Matching with Grover: Oracle Designs, QRAM Impact, and Complexity Analysis

What QRAM Promises

An ideal QRAM can, in principle:

- Take an address superposition $\sum_i \alpha_i |i\rangle$ and return

$$\sum_i \alpha_i |i\rangle |\text{data}[i]\rangle$$

  in $\mathcal{O}(\log n)$ or even $\mathcal{O}(1)$ time per query (depending on the model).

- This means you can load the substring for any address on demand during the oracle evaluation, without precomputing all possibilities.

Complexity Impact with QRAM

With QRAM:

- **State preparation:** Just apply Hadamards to `idx` $\rightarrow \mathcal{O}(\log n)$.
- **Oracle:** For each Grover iteration:
  - Query QRAM to load substring for the current address $\rightarrow \mathcal{O}(L)$ (or $\mathcal{O}(1)$ if you assume block loading).
  - Compare with patterns:
    - Naive: $\mathcal{O}(m \cdot L)$ equality checks.
    - Optimized (inner Grover over patterns): $\mathcal{O}(L\sqrt{m})$.
- **Grover iterations:** $\mathcal{O}(\sqrt{n/t})$.

So total complexity becomes:

With QRAM (inner Grover): $\boxed{\mathcal{O}\left(L\sqrt{m}\sqrt{\frac{n}{t}}\right)}$, With QRAM (enumerate m): $\boxed{\mathcal{O}\left(mL\sqrt{\frac{n}{t}}\right)}$,

instead of:

$$\text{Without QRAM: } \boxed{\mathcal{O}(nL)} + \boxed{\mathcal{O}\left(mL\sqrt{\frac{n}{t}}\right)}.$$

Practical Limits and Asymptotic Comparison

- **Without QRAM:** You pay $\mathcal{O}(nL)$ upfront $\rightarrow$ no speedup over Aho–Corasick (AC).
- **With QRAM:** You remove the $\mathcal{O}(nL)$ loader cost and get a quadratic speedup in n (and partial in m if you use inner Grover).
- This is the only scenario where the quantum algorithm can theoretically beat AC for the decision problem (existence of a match) when matches are rare (small t).
- QRAM is not available on current hardware and is considered one of the hardest open engineering problems in quantum computing.
- Even if QRAM existed, the algorithm still has:
  - Large constants (due to equality checks and Grover iterations).
  - Output cost: reporting t matches requires $\Omega(t)$ classical steps, so for large t, AC remains competitive.

Notation:

$n =$ text length, $L =$ pattern length (assume fixed), $m =$ number of patterns, $S = n-L+1 \approx n$, $t =$ nu

Asymptotic Comparison Table

Method	Preprocess / Build	Decision (search) time	Reporting overhead	Total (reporting) time	Space / Notes
Aho–Corasick (AC)	$\mathcal{O}\big(\sum_i L_i\big)$	$\mathcal{O}(n)$	$+\ \mathcal{O}(t)$	$\mathcal{O}(n+t)$	Linear in automaton size; practical gold standard
Quantum (no QRAM)	—	Loader $\mathcal{O}(nL)$ + Oracle $\mathcal{O}\big(mL\sqrt{n/t}\big)$	$+\ \mathcal{O}(t)$	$\mathcal{O}(nL)$ + $\mathcal{O}\big(mL\sqrt{n/t}\big)$ + $\mathcal{O}(t)$	Qubits $\sim \log n + 2L +$ and
Quantum with QRAM (enumerate m)	—	$\mathcal{O}\big(mL\sqrt{n/t}\big)$	$+\ \mathcal{O}(t)$	$\mathcal{O}\big(mL\sqrt{n/t}\big)$ + $\mathcal{O}(t)$	Assumes low-latency QRAM
Quantum with QRAM + inner Grover	—	$\mathcal{O}\big(L\sqrt{m}\sqrt{n/t}\big)$	$+\ \mathcal{O}(t)$	$\mathcal{O}\big(L\sqrt{m}\sqrt{n/t}\big)$ + $\mathcal{O}(t)$	Extra qubits; assumes QRAM

Plug-in Examples (Ignoring Constants)

Assume: $n = 10^6,\ m = 100,\ L = 10.$

Scenario A: Very Sparse Matches ($t = 1$)

$$\sqrt{n/t} = \sqrt{10^6} = 1000.$$

Method	Approx work (decision)	Add reporting
Aho–Corasick	$n \approx 1{,}000{,}000$	$+t = 1 \Rightarrow \sim 1{,}000{,}001$
Quantum (no QRAM)	Loader $nL \approx 10{,}000{,}000$ + Oracle $mL\sqrt{n/t} \approx 1{,}000{,}000$	$\sim 11{,}000{,}001$
Quantum + QRAM (enumerate m)	$mL\sqrt{n/t} \approx 1{,}000{,}000$	$\sim 1{,}000{,}001$
Quantum + QRAM + inner Grover	$L\sqrt{m}\sqrt{n/t} \approx 100{,}000$	$\sim 100{,}001$

Scenario B: Moderate Matches ($t = 1000$)

$$\sqrt{n/t} = \sqrt{10^6/10^3} = \sqrt{10^3} \approx 31.62.$$

Method	Approx work (decision)	Add reporting
Aho–Corasick	$n \approx 1{,}000{,}000$	$+t = 1000 \Rightarrow \sim 1{,}001{,}000$
Quantum (no QRAM)	Loader $nL \approx 10{,}000{,}000$ + Oracle $mL\sqrt{n/t} \approx 31{,}600$	$\sim 10{,}032{,}600$
Quantum + QRAM (enumerate m)	$mL\sqrt{n/t} \approx 31{,}600$	$\sim 32{,}600$
Quantum + QRAM + inner Grover	$L\sqrt{m}\sqrt{n/t} \approx 3162$	~ 4162

Observation. Decision cost can look favorable with ideal QRAM (especially with inner Grover), but the $+t$ reporting term grows; for larger t (or when constants/gate depth matter), AC's simple $\mathcal{O}(n+t)$ scan is hard to beat in practice.

Problem Context

We search over starting positions $i \in \{0, \ldots, S-1\}$ (with $S \approx n$) and mark those for which the L-mer $\texttt{text}[i : i+L]$ equals any of the m patterns in a set P. The outer Grover runs over the address register and needs a phase oracle

$$O_{\text{addr}} : \ |i\rangle\,|\text{data}_i\rangle\,|-\rangle \ \longmapsto \ (-1)^{[\,\text{data}_i \in P\,]}\ |i\rangle\,|\text{data}_i\rangle\,|-\rangle\,. \tag{1}$$

The difference below is how we implement $[\,\text{data}_i \in P\,]$.

1 Enumerate-m Oracle (Classical-Style OR)

Idea

Check all patterns one-by-one, and flip phase if any matches.

Mechanics

- **Registers**: `idx` (addresses), `data` (L-mer), `match`($|-\rangle$).
- For each pattern p_j ($j = 1, \ldots, m$):
  1. Map equality `data` $== p_j$ into "all-ones" on `data` (apply X on zero bits of p_j).
  2. Apply a multi-controlled X (MCX) from all `data` bits to `match`.

3. Uncompute the X's.

- If at least one equality holds, the `match` qubit flips (adds a -1 phase).

Cost per Outer Iteration

$$\Theta(m \cdot L) \text{ equality work:} \quad m \text{ patterns} \times \mathcal{O}(L) \text{ bit checks.}$$

Requires no pattern index register or pattern QRAM; patterns are hard-coded into the oracle.

Total Complexity (with QRAM for Text)

Outer Grover iterations: $\Theta\left(\sqrt{S/t}\right)$. Total:

$$\boxed{\Theta\left(m\,L\,\sqrt{\frac{S}{t}}\right)}.$$

Pros

- Simple circuit.
- Fewer qubits (no `pid` nor `pdat` registers).

Cons

- Scales linearly in m; for large m, the oracle dominates.
- If the pattern list has duplicates, phase flips can cancel mod 2; deduplicate first.

When to Use

- m is small (a handful to a few tens).
- You prefer simpler circuits and fewer qubits.

2 Inner Grover (Quantum OR Across Patterns)

Idea

Grover-search the pattern index space to test if $\exists\, k$ such that `pattern`$[k]$ == `data`.

Mechanics

- **Registers**: `idx` (addresses), `data` (L-mer), `pid` (pattern index), `pdat` (pattern data), `match`($|-\rangle$).
- Inner loop:
  1. Prepare `pid` in uniform superposition.
  2. Pattern QRAM (ideal) loads `pattern[pid]` into `pdat`.
  3. **Inner oracle**: flip phase if `data` == `pdat` (equality comparator).
  4. Apply inner diffuser on `pid`.
  5. Repeat $\left\lceil \frac{\pi}{4}\sqrt{m} \right\rceil$ times.

Cost per Outer Iteration

Inner Grover: $\Theta(\sqrt{m})$ iters; Equality per inner iter: $\Theta(L)$; $\Rightarrow$ Net: $\boxed{\Theta\big(L\sqrt{m}\big)}$.

Total Complexity (with QRAM for Text & Patterns)

Outer Grover iterations: $\Theta\Big(\sqrt{S/t}\Big)$. Total:

$$\boxed{\Theta\left(L\sqrt{m}\,\sqrt{\frac{S}{t}}\right)}.$$

Pros

- Asymptotically better in m ($\sqrt{m}$ vs. m).
- Appropriate when m is large and extra registers/QRAM are available.

Cons

- Needs more qubits: `pid` $\approx \lceil \log_2 m \rceil$, `pdat` $= 2L$, comparator ancillas ($\approx 2L$), MCX ancillas.
- Assumes pattern QRAM (or loader), plus inner diffuser.
- Larger constants and depth.

When to Use

- m is large (hundreds to thousands+).
- You target theoretical speedups (ideal QRAM).
- You can provision extra registers and tolerate deeper circuits.

Side-by-Side Summary

Aspect	**Enumerate-m**	**Inner Grover**
Oracle principle	Classical-style OR over all m patterns	Quantum OR via Grover over pattern IDs
Per outer iteration cost	$\Theta(m \cdot L)$	$\Theta(L\sqrt{m})$
Total (with QRAM)	$\Theta\big(m\,L\,\sqrt{S/t}\big)$	$\Theta\big(L\sqrt{m}\,\sqrt{S/t}\big)$
Registers	`idx`, `data`, `match`	`idx`, `data`, `pid`, `pdat`, `match` (+ ancillas)
QRAM needs	Text QRAM (ideal)	Text + Pattern QRAM (ideal)
Qubit footprint	Lower	Higher (extra `pid`, `pdat`, $\approx 2L$ eq-ancillas)
Best for	Small m, simpler circuits	Large m, theory-driven $\sqrt{m}$ speedup

Practical Implications

- **Without QRAM**, both approaches are swamped by $\mathcal{O}(n \cdot L)$ to coherently load substrings. Use demo circuits on tiny instances only.
- **With ideal QRAM**, inner Grover yields a $\sqrt{m}$ improvement over enumerate-m, but needs more qubits and depth.
- **Duplicates**: Deduplicate patterns. In enumerate-m, duplicate flips can cancel. In inner Grover, duplicates change the marked fraction and iteration tuning.
- **Equality oracle**: Use compute–oracle–uncompute to keep phase-only and avoid garbage entanglement with the outer loop.
- Unknown t: Use fixed-point Grover.

Code details

Enumerate-m:

- Builder: `build_qram_enumerate_m_circuit(...)`
- Oracle: `oracle_enumerate_patterns(...)` (explicit X/MCX per pattern)
- Per-iteration cost counters: `enumerate_pattern_equality_checks` $\approx L$ per pattern $\Rightarrow \Theta(m \cdot L)$

Inner Grover:

- Builder: `build_qram_inner_grover_circuit(...)`
- Pattern space: `pid` + `pdat`; inner diffuser via `diffuser_on_register(pid, ...)` (MCX/H chains)
- Equality oracle: `equality_flip_on_match(...)` using $2L$ XNOR ancillas + one MCX
- Per-iteration cost counters: `inner_iters` $\approx \lceil (\pi/4)\sqrt{m} \rceil$, `inner_oracle_equality_calls`, and gate counts reflecting $\Theta(L\sqrt{m})$

Quick Intuition

- **Enumerate-m**: "check every pattern; if any matches, flip." Simple; linear in m.
- **Inner Grover**: "Grover on pattern IDs to find a matching pattern faster." Replaces classical OR over m with a quantum OR in $\sqrt{m}$ steps, at the cost of more registers and QRAM.

Python Reference Implementations (Ideal-QRAM Simulators)

- **Quantum + QRAM (enumerate m).**
- **Quantum + QRAM + inner Grover (over pattern IDs).**

These are complexity-faithful simulators (not gate-level Qiskit circuits). They:

- Implement Grover amplitude amplification over the address space.

- Use ideal QRAM assumptions to avoid the $\mathcal{O}(n \cdot L)$ loader.
- Count oracle work to reflect the asymptotic costs:
  - Variant 1: per-iteration oracle cost $\Theta(mL)$.
  - Variant 2: per-iteration oracle cost $\Theta(L\sqrt{m})$ due to the inner Grover over patterns.
- Sample outcomes from the resulting probability distribution (so you get histograms similar to counts).

Why not Qiskit here? Physical QRAM does not exist. The code treats QRAM as an $\mathcal{O}(1)/\mathcal{O}(L)$ black box and models amplitude evolution and cost accounting. This matches the table's complexity rather than incurring an artificial $\mathcal{O}(nL)$ loader cost.

How These Implementations Reflect the Complexity Table

- **No $\mathcal{O}(n \cdot L)$ loader:** We assume ideal QRAM, so we don't build a heavy "QRAM-like" circuit. This aligns with the "with QRAM" rows in the table.
- **Per-iteration oracle cost:**
  - Variant 1 records `oracle_cost_equality_checks` $= r \cdot m \cdot L$, i.e., $\Theta(mL)$ per iteration over r iterations.
  - Variant 2 records `oracle_cost_equality_checks` $= r \cdot \lceil (\pi/4)\sqrt{m} \rceil \cdot L$, i.e., $\Theta(L\sqrt{m})$ per iteration over r iterations.
- **Number of Grover iterations:** Set to
$$r \approx \left\lfloor \frac{\pi}{4}\sqrt{\frac{S}{t}} \right\rfloor$$
via `choose_grover_iterations`, consistent with the table.
- **Output cost:** The simulators return sampled matches; in real systems you still pay $\Omega(t)$ to output them.

Qiskit usage to demo QRAM-like Qiskit skeleton circuits that:

- Treat the "QRAM query" as a custom oracle instruction (black box).
- Implement the outer diffuser on the address register.
- Use the same cost accounting to mirror the asymptotics—without explicitly materializing an $\mathcal{O}(nL)$ loader. This helps track qubit counts and depth.

Theory / Scalable Skeletons (Ideal QRAM)

- Circuits with **opaque QRAM nodes** (compute–oracle–uncompute).
- Explicit (non-opaque) **inner Grover diffuser** on `pid` and an explicit **equality test** (`data` == `pdat`) using MCX/H chains.
- **Cost counters** (per iteration) to match the intended asymptotic costs:
  - Enumerate-m: $\Theta(m \cdot L)$ per Grover iteration.
  - Inner Grover: $\Theta(L \cdot \sqrt{m})$ per Grover iteration.

Practical Demo (Small n, L, m) with QRAM-like Loaders

- Replaces ideal QRAM with **coherent QRAM-like loaders** (e.g., an efficient loader) for text and pattern registers.
- Explicit **inner oracle** (equality) and **inner diffuser** on `pid`.
- **Simulatable** in Aer for toy sizes, but **breaks the ideal QRAM complexity** by adding an $\mathcal{O}(n \cdot L)$ loader cost.

In general, this effort has theoretical implementation in an ideal QRAM as a part of a readiness effort for the fault-tollerant era. Also there's a demo for NISQ SDK aka. Qiskit, to showcase qram algorithm that in big instances wins, asymptotically the well suited Aho Corasick.